

TypingRaceSimulator — Part 1 Report

1. Overview

This project implements a textual typing race simulation using object-oriented programming in Java. Three typists compete to complete a passage of text by advancing character by character. The simulation includes realistic behaviour such as typing accuracy, mistakes (slide back), and burnout.

The program is structured into two main classes:

- * `Typist` — represents an individual typist and their state
- * `TypingRace` — manages the race simulation and game logic

2. Design and Structure

2.1 Typist Class

The `Typist` class encapsulates all data and behaviour related to a typist.

Fields

The class contains six private fields:

- * `String name` — typist's name
- * `char symbol` — visual representation in the race
- * `int progress` — current position in the race
- * `boolean burntOut` — whether the typist is burnt out
- * `int burnoutTurnsRemaining` — remaining burnout duration
- * `double accuracy` — typing accuracy (0.0 to 1.0)

Encapsulation is enforced by keeping all fields private and providing controlled access via methods.

2.2 Key Methods

Progress Control

- * `typeCharacter()` increases progress by 1 if not burnt out
- * `slideBack(int amount)` reduces progress but never below 0

Burnout System

- * `burnOut(int turns)` activates burnout
- * `recoverFromBurnout()` reduces burnout duration each turn
- * `isBurntOut()` checks current state

Accuracy Handling

- * `setAccuracy(double)` ensures values stay between 0.0 and 1.0

Reset Functionality

- * `resetToStart()` resets progress and clears burnout state

2.3 TypingRace Class

The `TypingRace` class controls the simulation.

Features:

- * Supports three typists (seats 1–3)
- * Runs a turn-based simulation loop
- * Applies randomness using `Math.random()`
- * Displays race progress in the terminal

Game Mechanics:

- * Typing success depends on accuracy
- * Mistypes cause the typist to move backwards
- * Burnout temporarily prevents typing

3. Implementation Details

Several improvements and fixes were made to the original starter code:

- * Ensured all typists are reset at the start of the race
- * Fixed race completion condition using `>=` instead of `==`
- * Adjusted mistype probability so higher accuracy reduces mistakes
- * Implemented winner detection and output

* Replaced unsupported Unicode symbols with standard characters for compatibility

4. Testing

4.1 Typist Testing

A separate test class (`TypistTest`) was created to verify functionality.

Tests performed:

- * Progress increases correctly when typing
- * Progress does not go below zero when sliding back
- * Burnout activates and recovers correctly
- * Accuracy is correctly clamped between 0.0 and 1.0
- * Reset clears progress and burnout state

All tests produced expected outputs.

4.2 TypingRace Testing

A second test (`TypingRaceTest`) was used with a shorter race length.

Verified behaviour:

- * Race runs correctly with multiple typists
- * Burnout states are displayed
- * Progress updates visually in real time
- * Winner is correctly determined

5. Reflection

This project demonstrates key object-oriented programming concepts:

- * **Encapsulation** — using private fields and public methods
- * **Abstraction** — separating typist behaviour from race logic
- * **State management** — tracking progress and burnout
- * **Modularity** — separating functionality into classes

Challenges included:

- * Implementing correct burnout logic
- * Ensuring progress never becomes negative
- * Balancing randomness for realistic behaviour

Overall, the final program behaves as expected and meets the requirements of the specification.

6. Conclusion

The Typing Race Simulator successfully models a competitive typing environment using object-oriented principles. The program is robust, tested, and demonstrates correct handling of edge cases such as burnout and mistypes.

7. Encapsulation Explanation

Encapsulation is the principle of restricting direct access to an object's internal data and only allowing interaction through controlled methods. In the Typist class, all fields are declared private, ensuring they cannot be modified directly from outside the class.

Accessor methods (getters):

- getName()
- getSymbol()
- getProgress()
- getAccuracy()
- getBurnoutTurnsRemaining()
- isBurntOut()

These methods allow safe read-only access to internal state.

Mutator methods (setters):

- setAccuracy(double newAccuracy)
- setSymbol(char newSymbol)

These methods allow controlled modification of fields.

Validation is implemented to ensure correctness:

- In setAccuracy(), values are clamped between 0.0 and 1.0 to prevent invalid accuracy values.

- In `slideBack(int amount)`, progress is prevented from going below zero, ensuring the typist cannot move before the start of the race.

This protects the integrity of the object and prevents invalid states.

8. Testing Evidence Explanation

Several tests were conducted to verify the correctness of the `Typist` class.

Test 1: `typeCharacter()`

This test confirms that progress increases correctly when a character is typed.

Test 2: `slideBack()`

This ensures that progress does not go below zero, even when a large slide amount is applied.

Test 3: Burnout logic

This verifies that burnout activates correctly and that the burnout counter decreases each turn until the typist recovers.

Test 4: Accuracy lower bound

This confirms that accuracy values below 0.0 are clamped to 0.0.

Test 5: Accuracy upper bound

This confirms that accuracy values above 1.0 are clamped to 1.0.

Test 6: `resetToStart()`

This ensures that both progress and burnout state are reset correctly.

All tests produced the expected results, confirming correct behaviour.

9. TypingRace Bug Fixes and Improvements

Several issues were identified in the original `TypingRace` class:

Issue 1: Not all typists reset

Cause: Only two typists were reset at the start of the race.

Fix: Added `resetToStart()` for all three typists.

Issue 2: Incorrect win condition

Cause: The race only ended when `progress == passageLength`.

Problem: If a typist overshoots (e.g. goes from 39 to 41), they would not win.

Fix: Changed condition to `progress >= passageLength`.

Issue 3: Mistype probability incorrect

Cause: Mistype chance increased with accuracy.
Problem: More accurate typists should make fewer mistakes.
Fix: Changed probability to $(1 - \text{accuracy}) * \text{baseChance}$.

Issue 4: No winner displayed

Cause: `startRace()` did not print the winner.

Fix: Added logic to determine and display the winning typist.

Issue 5: Unicode symbols not displaying

Cause: Terminal did not support special characters.

Fix: Replaced with standard characters ('1', '2', '3').

These fixes improve correctness, realism, and usability of the simulation.

10. Evidence

Figure 1: TypistTest Output

```
>> java TypistTest
Test 1: typeCharacter moves progress forward
Expected: 1
Actual: 1

Test 2: slideBack cannot go below zero
Expected: 0
Actual: 0

Test 3: burnout counts down and clears
Burnt out? true
Turns left after 1 recover: 1
Burnt out after 2 recovers? false

Test 4: accuracy clamps below 0
Expected: 0.0
Actual: 0.0

Test 5: accuracy clamps above 1
Expected: 1.0
Actual: 1.0

Test 6: resetToStart clears progress and burnout
Progress expected: 0, actual: 0
Burnt out expected: false, actual: false
Burnout turns expected: 0, actual: 0
PS C:\Users\kunal\OneDrive\Email attachments\Documents\TypingRaceSimulator\Part1>
```

Explanation:

This figure shows the results of unit tests for the Typist class. It confirms that:

- Progress increases correctly when typing
- Progress does not go below zero after slideBack()
- Burnout correctly counts down and clears
- Accuracy values are clamped between 0.0 and 1.0
- resetToStart() clears both progress and burnout state

Figure 2: TypingRaceTest Output (Short Race)

```

-----
TYPING RACE - length: 10
1 ----- FAST
2 - MEDIUM
3 SLOW
-----

TYPING RACE - length: 10
1 ----- FAST
2 -- MEDIUM
3 SLOW
-----

Winner: FAST
PS C:\Users\kunal\OneDrive\Email attachments\Documents\TypingRaceSimulator\Part1>

```

Explanation:

This figure shows a short race simulation using three typists with different accuracy levels. The output demonstrates:

- Progress increasing over time for each typist
- Differences in speed due to varying accuracy values
- Correct race termination when a typist reaches the passage length
- The winner (“FAST”) being correctly identified and displayed

Figure 3: Full TypingRace Simulation

```

TYPING RACE - length: 40
1 ----- TURBOFINGERS
2 ----- QWERTY_QUEEN
3 ---- HUNT_N_PECK
-----

TYPING RACE - length: 40
1 ----- TURBOFINGERS
2 ----- QWERTY_QUEEN
3 ---- HUNT_N_PECK
-----

TYPING RACE - length: 40
1 ----- TURBOFINGERS
2 ----- QWERTY_QUEEN
3 ---- HUNT_N_PECK
-----

TYPING RACE - length: 40
1 ----- TURBOFINGERS
2 ----- QWERTY_QUEEN
3 --- HUNT_N_PECK
-----

TYPING RACE - length: 40
1 ----- TURBOFINGERS
2 ----- QWERTY_QUEEN
3 - HUNT_N_PECK
-----

TYPING RACE - length: 40
1 ----- TURBOFINGERS
2 ----- QWERTY_QUEEN
3 - HUNT_N_PECK
-----

Winner: TURBOFINGERS

```

Explanation:

This figure shows the full typing race simulation with a passage length of 40 characters. It demonstrates:

- Continuous progress updates for all typists

- Differences in speed based on accuracy
- Dynamic race behaviour over multiple turns
- Correct detection and display of the winning typist ("TURBOFINGERS")